MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Maxim Ormanji FAF-242

Report

Laboratory work No.1

Software Design Techniques and Mechanisms

Topic: Implement 3 SOLID letters in a simple project

Checked by:

Istrati Daniel, *university assistant*

DISA, FCIM, UTM

Chișinău – 2026

CONTENTS

INTRODUCTION **2**

1 Theoretical Background **3**

1.1 SOLID Principles 3

1.2 Principles Selected for This Work 3

2 Program and Results **4**

2.1 Project Description 4

2.2 Applied SOLID Principles 5

2.3 Source Code 6

2.4 Program Output 8

2.5 Extending the Program (Open/Closed Demonstration) 9

CONCLUSION **11**

BIBLIOGRAPHY **12**

INTRODUCTION

As a program grows, every new requirement forces the programmer to change existing code, and every change risks breaking something that already works. Object-oriented design offers a set of guidelines that make such changes cheaper and safer. The most widely used of them are the five SOLID principles, formulated by Robert C. Martin.

This laboratory work applies selected SOLID principles in a small Java application: a coffee shop ordering system. The user chooses drinks from a menu, selects a payment method, and receives a receipt. The program is deliberately simple, so that each principle can be seen clearly in a few short classes.

1 THEORETICAL BACKGROUND

1.1 SOLID Principles

SOLID is an acronym for five design principles whose common goal is to make code easy to change: a modification in one place should not break other places. Each principle addresses a concrete, recognisable problem in the code.

1. **S – Single Responsibility Principle.** A class should have only one reason to change, that is, one job. A class that both calculates an order total and prints a receipt must be modified whenever either of those two things changes.

2. **O – Open/Closed Principle.** Software entities should be open for extension but closed for modification. New behaviour is added by writing new classes, not by editing working ones; a typical warning sign is a long `if / else if` chain over object types.

3. **L – Liskov Substitution Principle.** An object of a subclass must be usable wherever an object of its parent class is expected, without breaking the program. A subclass that throws an exception in an inherited method violates this rule.

4. **I – Interface Segregation Principle.** Many small, specific interfaces are better than one large general interface. A class should not be forced to implement methods it does not need.

5. **D – Dependency Inversion Principle.** High-level classes should depend on abstractions (interfaces), not on concrete low-level classes. The needed object is passed in from outside, for example through a constructor, instead of being created with `new` inside the class.

1.2 Principles Selected for This Work

Three principles are demonstrated explicitly in this project: **S**, **O** and **D**. The Liskov principle is respected implicitly, since every drink class can be used in place of the base class `Drink`. The Interface Segregation principle is not demonstrated specially; the only interface in the project, `PaymentMethod`, is kept minimal and contains a single method.

2 PROGRAM AND RESULTS

2.1 Project Description

The application is a console program. The user sees a numbered menu, adds drinks one by one, enters `0` to finish the order, chooses a payment method (cash or card), and receives a receipt. Invalid input is handled by repeating the question. The project consists of the classes listed in Table 2.1.

Table 2.1 – Classes of the coffee shop project

Class / interface	Responsibility
`Drink`	Abstract base class: stores the name and price of a drink
`Espresso`, `Latte`, `Cappuccino`	Concrete drinks; each extends `Drink` and sets its own name and price
`Menu`	Holds the list of drinks sold by the shop
`Order`	Stores the chosen drinks and calculates the total
`ReceiptPrinter`	Prints the receipt for an order
`PaymentMethod`	Interface with one method, `pay(double amount)`
`CashPayment`, `CardPayment`	Implementations of `PaymentMethod`
`Cashier`	Performs the checkout: takes the payment and prints the receipt
`Main`	Console input and output; connects all the parts

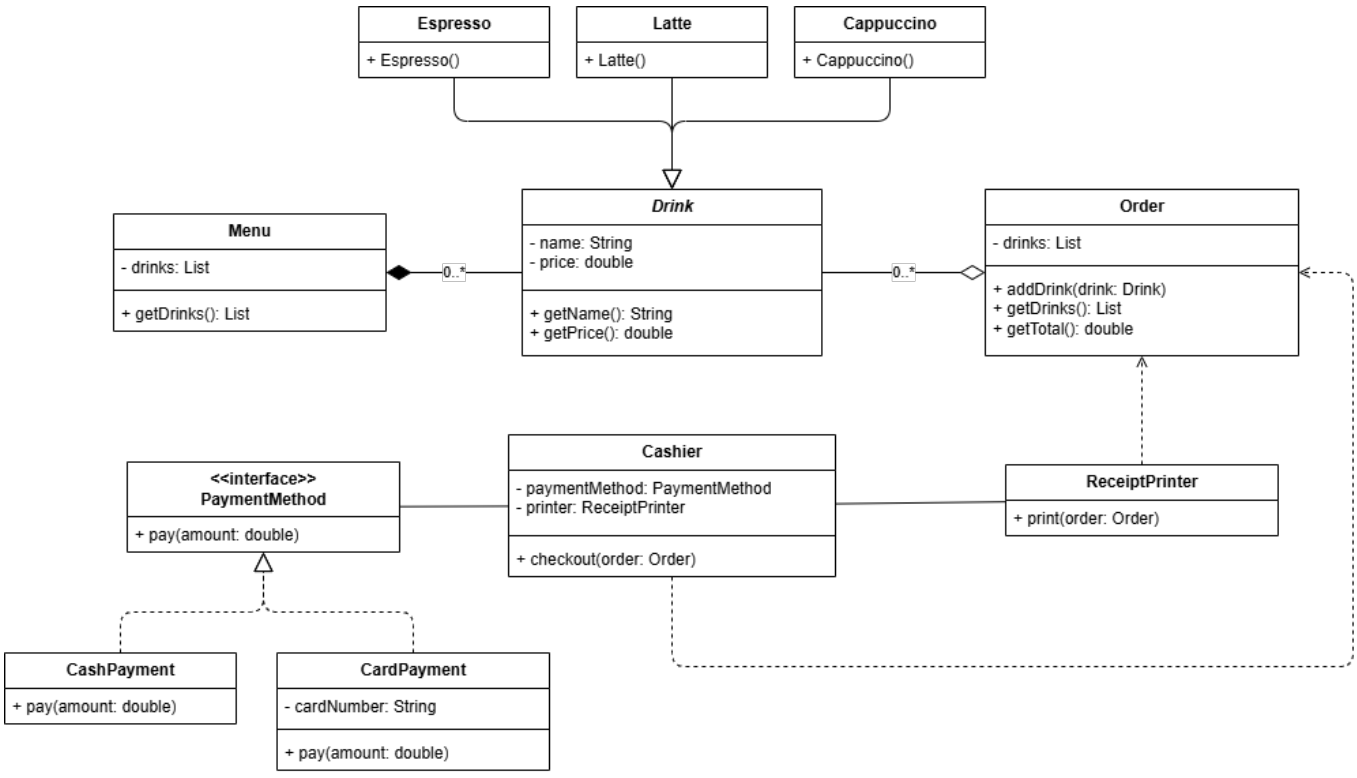


Figure 2.1. Class structure of the coffee shop system

2.2 Applied SOLID Principles

S – Single Responsibility

Every class in the project has exactly one job. The class `Order` only stores drinks and calculates the total; it does not print anything. Printing is the sole task of `ReceiptPrinter`. As a result, a change in the appearance of the receipt affects only `ReceiptPrinter`, and a change in the pricing logic affects only `Order` and the drink classes. In the same way, `Menu` only knows what the shop sells, and `Main` only communicates with the user.

O – Open/Closed

The principle is applied in two places.

Drinks. `Order`, `ReceiptPrinter` and `Cashier` work with the abstract type `Drink` and never check which concrete drink it is. A new drink, for example `Mocha`, is added by creating one new class that extends `Drink` and adding one line to `Menu`. No other class has to be modified. The menu in `Main` is printed from the list of drinks in a loop, so `Main` does not change either.

Payment methods. A new payment method is added by writing a new class that implements `PaymentMethod`; `Cashier` stays unchanged.

D – Dependency Inversion

The class `Cashier` does not depend on `CashPayment` or `CardPayment`. It depends only on the interface `PaymentMethod`, and the concrete implementation is passed through the constructor:

```
Cashier cashier = new Cashier(payment, new ReceiptPrinter());
```

The choice between cash and card is made in `Main` according to the user's input, and `Cashier` works identically with both. In the opposite design, with `new CashPayment()` written inside `Cashier`, supporting card payment would require rewriting `Cashier`.

2.3 Source Code

The complete source code of the project is available in the GitHub repository [3]. Only the key fragments that illustrate the applied principles are shown below.

Drink and its subclasses (O)

```
public abstract class Drink {
    private String name;
    private double price;

    public Drink(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public String getName() { return name; }
    public double getPrice() { return price; }
}

public class Latte extends Drink {
    public Latte() {
        super("Latte", 3.50);
    }
}
```

Order (S)

```
public class Order {
    private List<Drink> drinks = new ArrayList<>();

    public void addDrink(Drink drink) {
        drinks.add(drink);
    }

    public double getTotal() {
        double total = 0;
        for (Drink drink : drinks) {
            total = total + drink.getPrice();
        }
        return total;
    }
}
```

PaymentMethod and Cashier (O, D)

```
public interface PaymentMethod {
    void pay(double amount);
}

public class CashPayment implements PaymentMethod {
    @Override
    public void pay(double amount) {
        System.out.println("Paid " + amount + " $ in cash.");
    }
}

public class Cashier {
    private PaymentMethod paymentMethod;
    private ReceiptPrinter printer;

    public Cashier(PaymentMethod paymentMethod, ReceiptPrinter printer) {
        this.paymentMethod = paymentMethod;
        this.printer = printer;
    }

    public void checkout(Order order) {
```

```
        paymentMethod.pay(order.getTotal());
        printer.print(order);
    }
}
```

Main (fragments)

```
        // 1. The user chooses drinks until he enters 0
        boolean ordering = true;
        while (ordering) {
            System.out.println("Menu:");

            // The menu is printed from the list, so it is always up to date
            for (int i = 0; i < drinks.size(); i++) {
                Drink drink = drinks.get(i);
                System.out.println((i + 1) + " - " + drink.getName()
                        + " (" + drink.getPrice() + " $)");
            }
            ...
            if (choice == 0) {
                ordering = false;
            } else if (choice >= 1 && choice <= drinks.size()) {
                Drink chosen = drinks.get(choice - 1);
                order.addDrink(chosen);
            } else {
                System.out.println("Wrong choice, try again.");
            }
        }

        // 2. The user chooses how to pay
        PaymentMethod payment = null;
        while (payment == null) {
            ...
            if (choice.equals("1")) {
                payment = new CashPayment();
            } else if (choice.equals("2")) {
                ...
                payment = new CardPayment(cardNumber);
            }
        }

        // 3. Checkout (Cashier works with any PaymentMethod)
        Cashier cashier = new Cashier(payment, new ReceiptPrinter());
        cashier.checkout(order);
```

2.4 Program Output

Scenario 1: Payment in Cash

The user orders one Espresso and one Latte and pays in cash. The expected total is $2.5 + 3.5 = 6.0$.

```
Welcome to the Coffee Shop!

Menu:
1 - Espresso (2.5 $)
2 - Latte (3.5 $)
3 - Cappuccino (3.0 $)
0 - Finish order
Your choice: 1
Espresso added.

Menu:
1 - Espresso (2.5 $)
2 - Latte (3.5 $)
3 - Cappuccino (3.0 $)
0 - Finish order
Your choice: 2
Latte added.

Menu:
1 - Espresso (2.5 $)
2 - Latte (3.5 $)
3 - Cappuccino (3.0 $)
0 - Finish order
Your choice: 0

How do you want to pay?
1 - Cash
2 - Card
Your choice: 1

Paid 6.0 $ in cash.
===== COFFEE SHOP RECEIPT =====
Espresso - 2.5 $
Latte - 3.5 $
------------------------------
TOTAL: 6.0 $
==============================
```

Figure 2.2. Scenario 1 – order paid in cash

Scenario 2: Payment by Card

The user orders two Cappuccinos and one Espresso and pays by card. The expected total is $3.0 + 3.0 + 2.5 = 8.5$.

Scenario 3: Invalid Input

The user enters a number that is not in the menu and a text instead of a number. The program reports an error and shows the menu again.

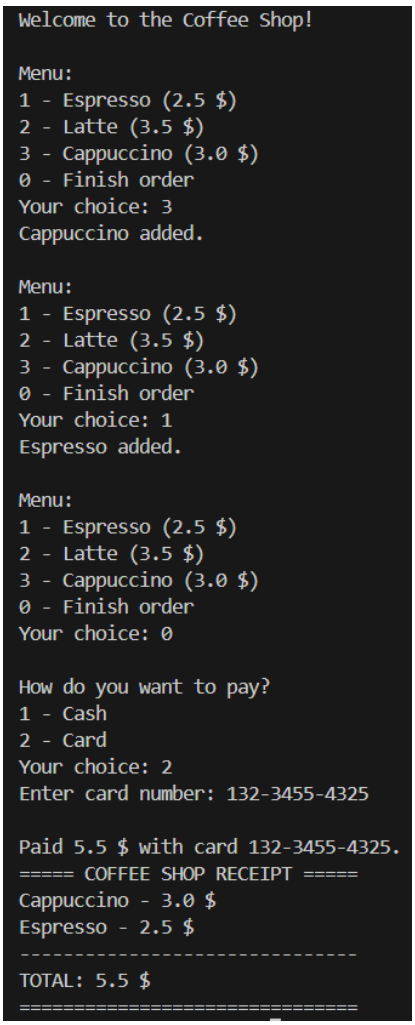


Figure 2.3. Scenario 2 – order paid by card

2.5 Extending the Program (Open/Closed Demonstration)

To demonstrate the Open/Closed principle, a new drink is added to the shop. One new class is created:

```
public class Mocha extends Drink {
    public Mocha() {
        super("Mocha", 4.00);
    }
}
```

and one line is added to the constructor of `Menu`:

```
drinks.add(new Mocha());
```

The classes `Main`, `Order`, `ReceiptPrinter` and `Cashier` remain unchanged, and the new drink appears in the menu as item 4 and works in the order and in the receipt.

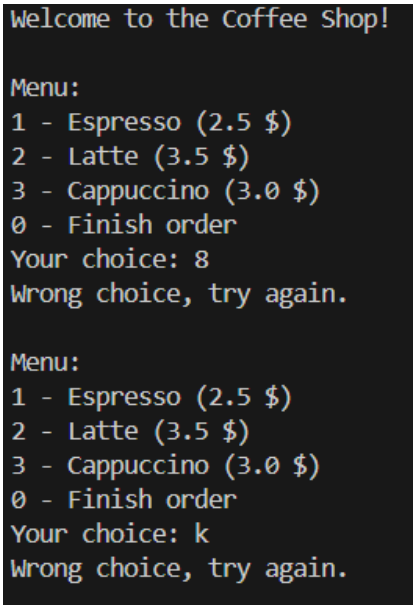


Figure 2.4. Scenario 3 – handling of invalid input

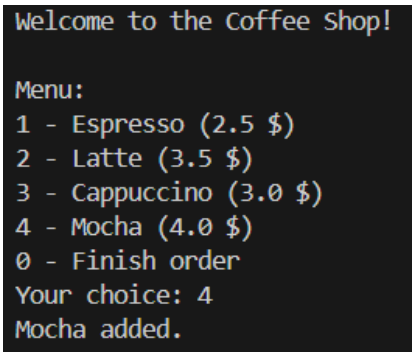


Figure 2.5. Menu after adding the `Mocha` class

CONCLUSION

This laboratory work demonstrated the practical use of the SOLID principles in a small Java project. The coffee shop system was divided into classes with clearly separated responsibilities, which illustrates the Single Responsibility principle: the order, the receipt, the menu and the user input are handled by different classes.

The Open/Closed principle was applied through the interface `PaymentMethod` and the abstract class `Drink`. New drinks and new payment methods are added as new classes, and the existing code does not have to be modified. The Dependency Inversion principle was applied in the class `Cashier`, which depends only on an interface (`PaymentMethod`) and receives a concrete implementation through its constructor. Therefore it works identically with cash and card payments.

The project also showed that these principles pay off as a program grows: the more drinks and payment methods the shop has, the more valuable it is that each new feature can be added without touching, and possibly breaking, the code that already works. In this project the `Cashier` still depends on the concrete class `ReceiptPrinter`; introducing an interface for the printer would be a natural next step to apply the Dependency Inversion principle fully.

BIBLIOGRAPHY

[1] Coffee shop system – source code of the project [online resource]. Available at: https://github.com/Orm-Max/tmps_repository (accessed: 2026).